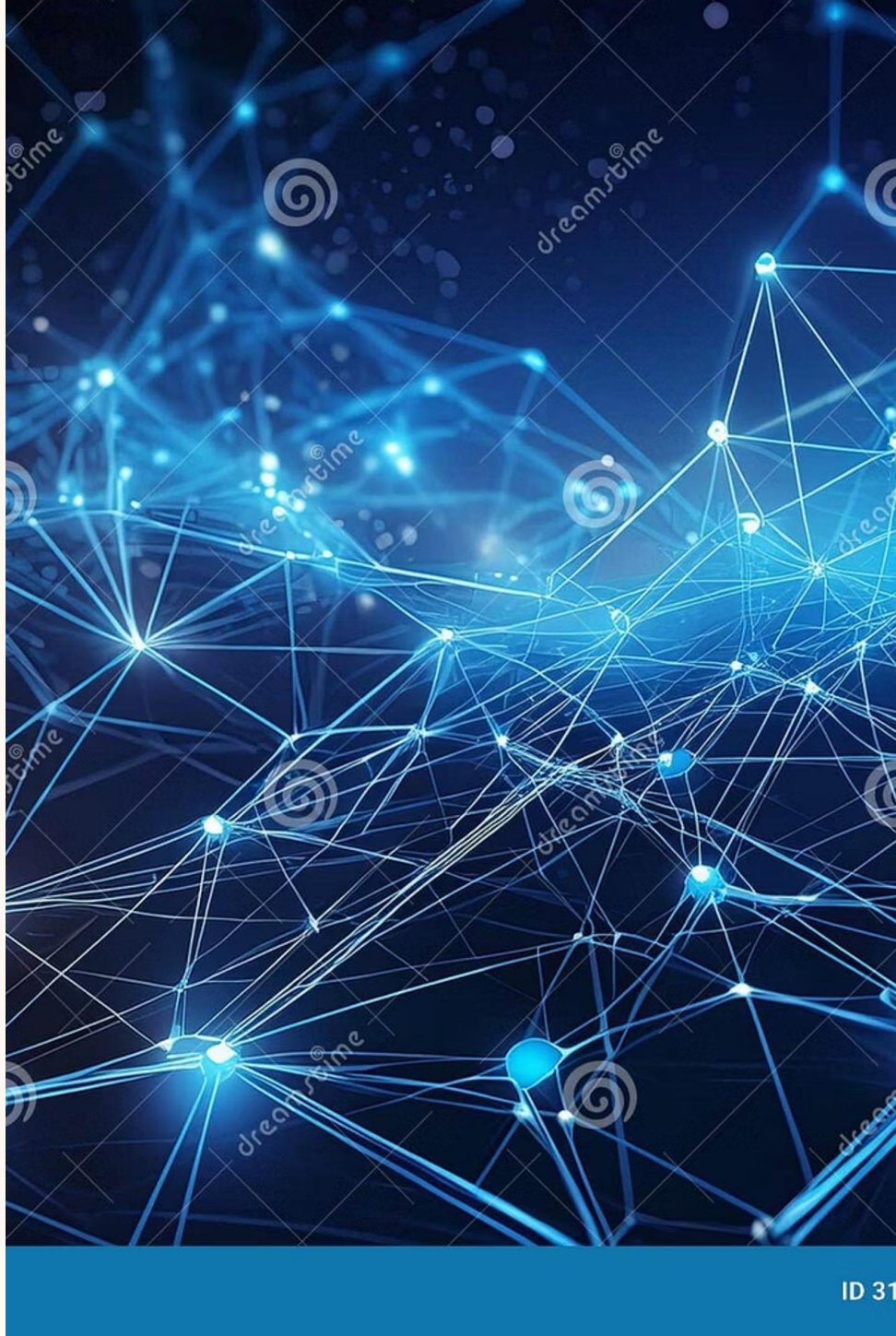
The Knuth-Morris-Pratt Algorithm

Smarter string searching through elegant algorithmic design

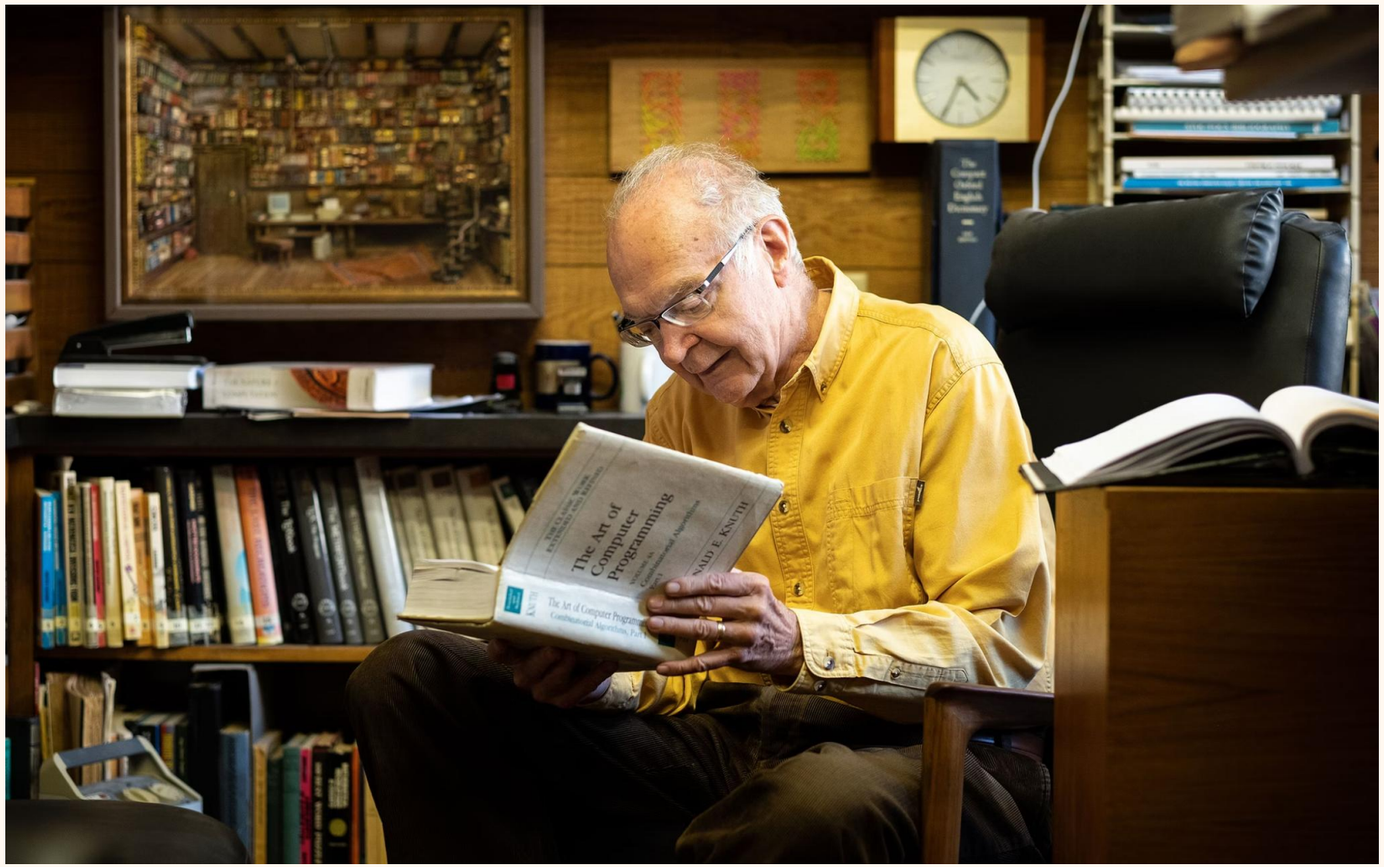


Finding Needles in Haystacks

Imagine searching for a specific word in a massive document — or a DNA sequence within an entire genome. Naive string matching algorithms are painfully slow, re-checking characters unnecessarily.

⚠ Worst case: searching a text of length **n** with a pattern of length **m** takes **O(n × m)** time — a significant bottleneck for large datasets.

The Breakthrough: KMP's Intelligent Leap



Developed in 1977 by **Donald Knuth**, **Vaughan Pratt**, and **James H. Morris**, KMP revolutionised string searching by achieving true linear time complexity.

The core insight: **use the pattern itself** to guide the search — avoiding redundant comparisons entirely.

Naive Approach

O(n × m)

KMP Algorithm

O(n + m)

The Secret Weapon: The LPS Array

The **Longest Proper Prefix which is also a Suffix** (LPS) array is the heart of KMP. For each position in the pattern, LPS stores the length of the longest prefix that is also a suffix of the substring ending at that position.

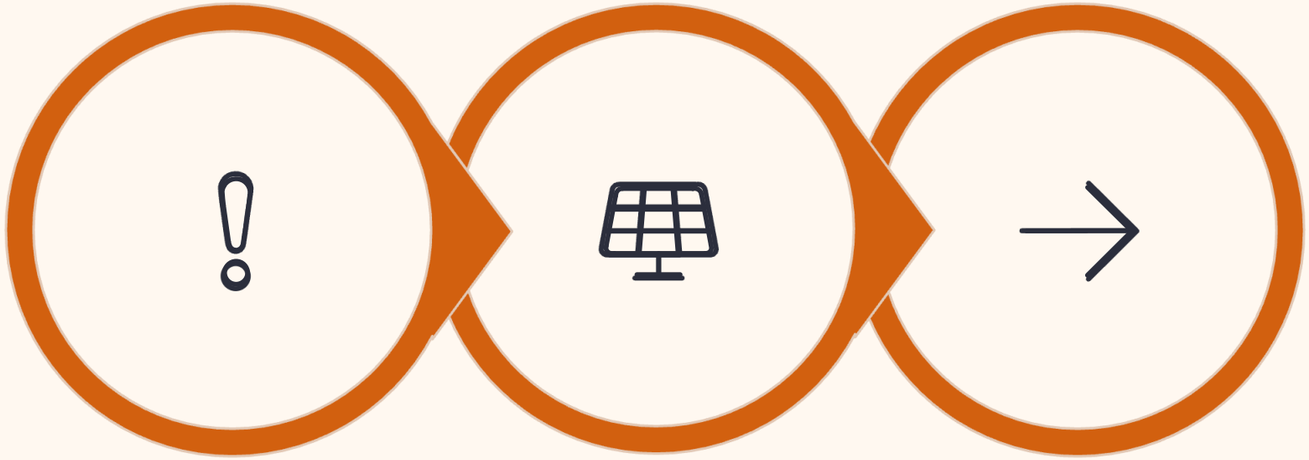
Example: Pattern `"ABABCABAB"`

Index	0	1	2	3	4	5	6	7	8
Char	A	B	A	B	C	A	B	A	B
LPS	0	0	1	2	0	1	2	3	4

Interpreting the LPS

For the prefix `"ABAB"`, the LPS value is **2** — meaning `"AB"` is both a proper prefix and a suffix. This tells KMP exactly how far to shift the pattern after a mismatch, without re-scanning the text.

How LPS Saves the Day



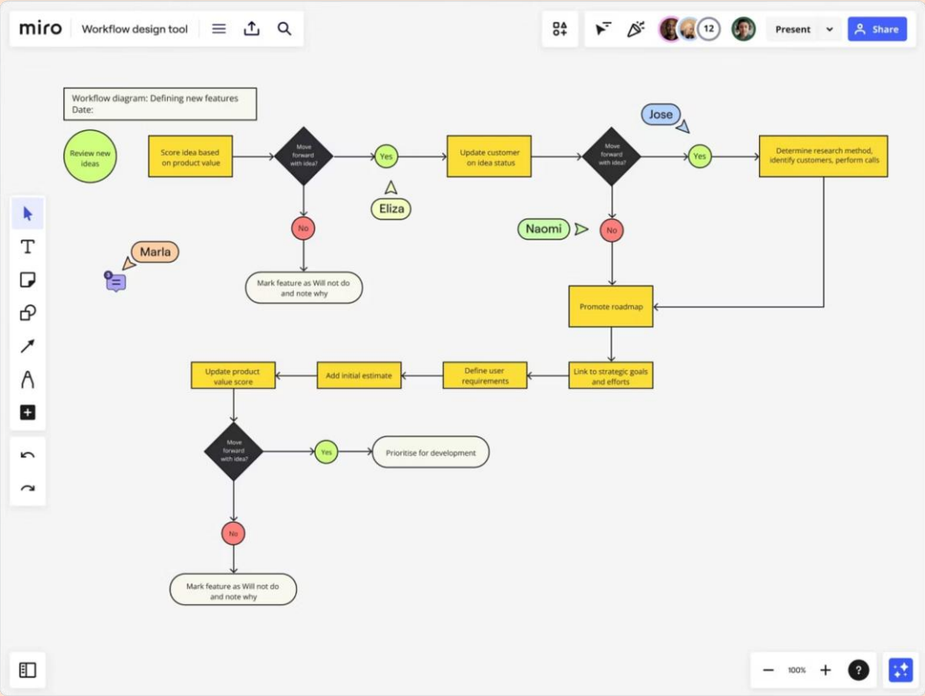
Mismatch Consult LPS Shift Pattern

When a mismatch occurs, instead of shifting the pattern by just one character, KMP consults the LPS array to skip ahead — guaranteeing that already-matched characters are never re-examined in the text.

Visualising the KMP Advantage

Text vs. Pattern

Text: `ABABD`



The Two Phases of KMP

1

Phase 1: Preprocessing

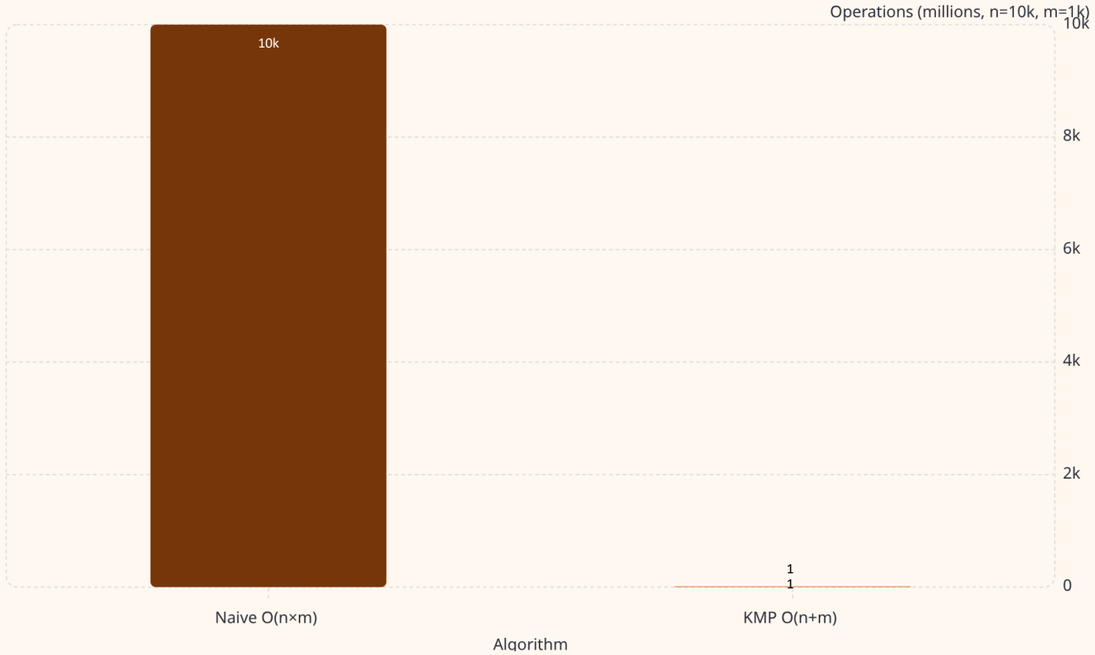
Compute the **LPS array** for the pattern. Analyses the pattern's internal structure in **O(m)** time.

2

Phase 2: Searching

Use the LPS array to scan the text efficiently, shifting intelligently on mismatches in **O(n)** time.

The Power of Linear Time



A Transformative Difference

O(n + m) means search time grows *linearly* — not quadratically — with input size. For large texts and patterns, this translates to orders-of-magnitude fewer operations.

KMP guarantees efficient, predictable performance **regardless** of input string characteristics — no pathological worst cases.

Real-World Impact

Text Editors

Powers the **Find & Replace** functionality in editors like VS Code and Sublime Text, delivering near-instant results across large files.

Bioinformatics

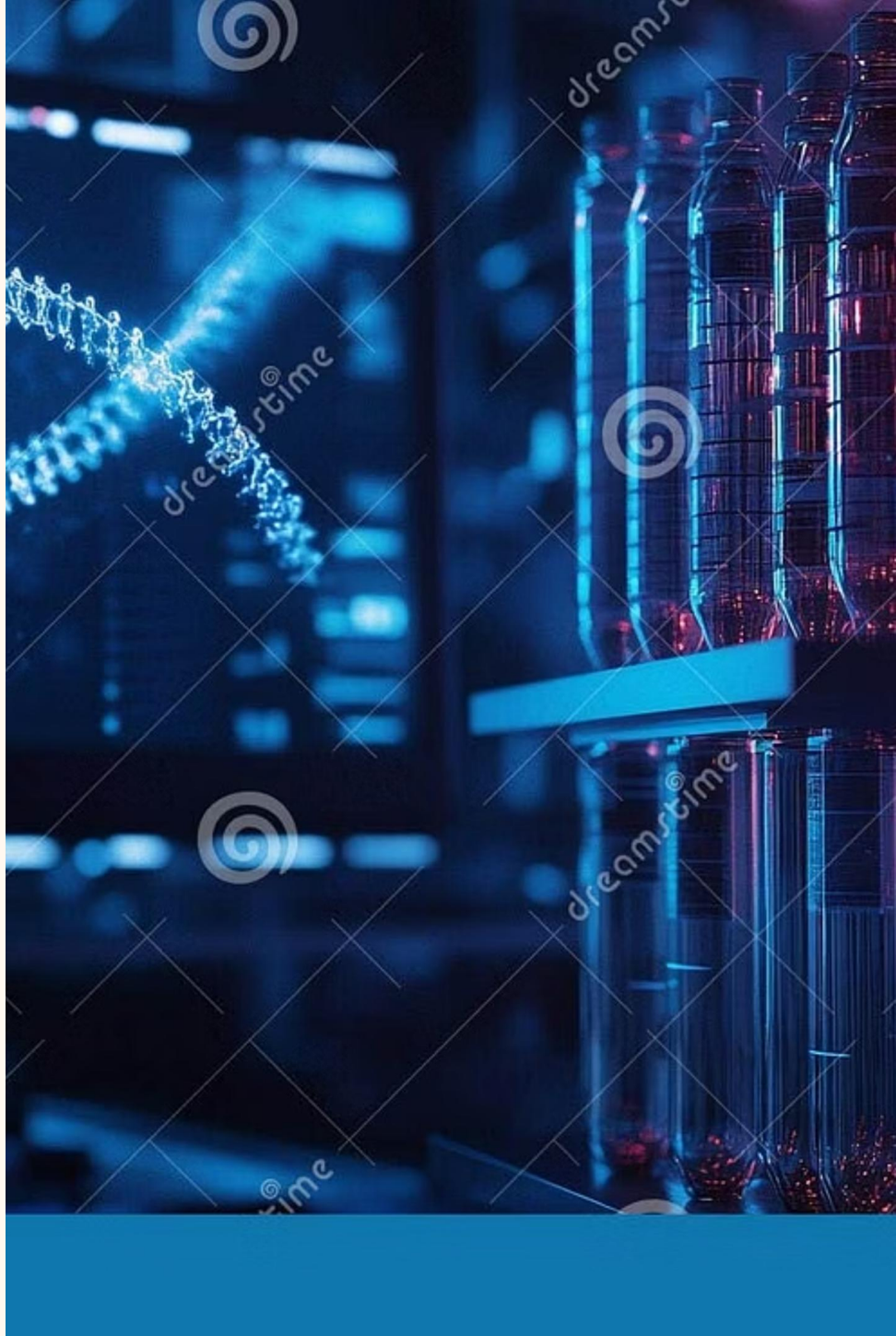
Essential for scanning vast genomic databases to locate specific DNA or protein sequences with clinical precision.

Network Security

Used in **intrusion detection systems** to rapidly identify malicious byte patterns within high-volume network traffic.

Data Compression

Forms the algorithmic basis for several compression techniques that rely on identifying repeated patterns within data streams.



KMP: The Smart Way to Search

> A testament to elegant algorithmic design — KMP proves that understanding structure is the key to unlocking efficiency.

Linear Complexity

O(n + m) guarantees fast, scalable performance on any input.

Zero Redundancy

No character in the text is ever examined more than once.

Pattern-Driven

The LPS array transforms the pattern into its own search guide.

From text editors to genomic research, **KMP remains a cornerstone of efficient string processing** in computer science.

Log Guardian: Applying KMP to Real-World Security

Log Guardian is a cybersecurity tool that scans massive server log files for malicious activity using the KMP algorithm. Rather than using slow, naive search methods that waste CPU power on redundant character comparisons, Log Guardian leverages KMP's linear time complexity to detect threats in real-time.

The Problem:

Traditional security tools use simple search methods that become painfully slow as log files grow into the gigabytes. A naive search algorithm might examine the same character thousands of times, creating bottlenecks in threat detection.

The Solution:

By implementing the KMP Algorithm with its precomputed LPS (Longest Prefix Suffix) Table, Log Guardian achieves O(n + m) linear time complexity. When a potential threat signature partially matches and then hits a mismatch, the algorithm doesn't restart—it knows exactly where to skip to based on the patterns already processed.

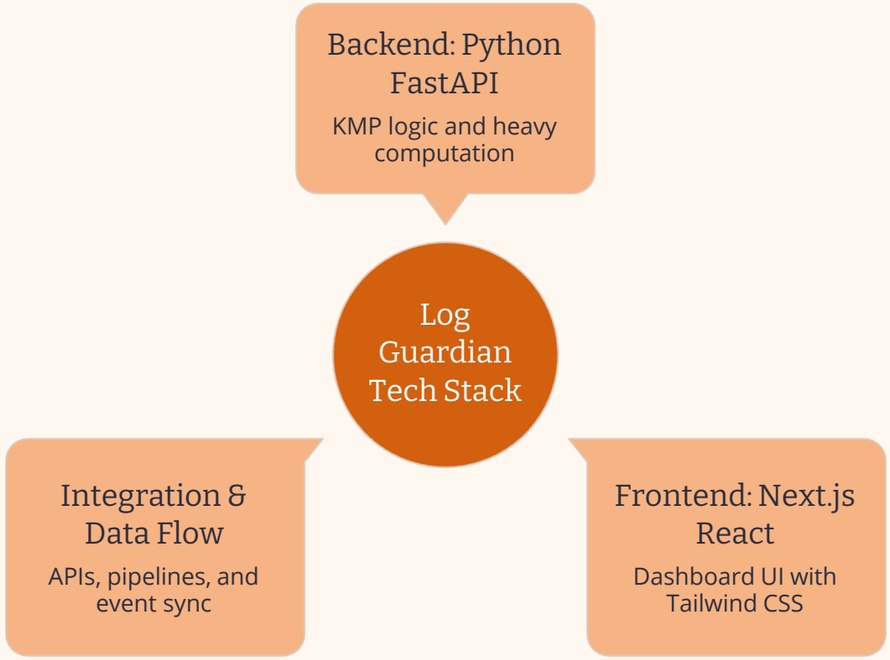
Log Guardian: Tech Stack & Threat Detection

Tech Stack

Backend: Python (FastAPI) — Implements the KMP algorithm logic and handles heavy computational lifting

Frontend: Next.js (React) with Tailwind CSS — Provides a clean, modern dashboard to visualize results

Integration: Multipart form submission sends log files to backend; backend returns JSON with threat count and execution time



This diagram illustrates Log Guardian's architecture, ensuring a robust, efficient, and user-friendly threat detection system.

Threat Detection Capabilities

The system scans for three major attack types:

- **SQL Injection:** Detects patterns like ' OR 1=1
- **XSS (Cross-Site Scripting):** Detects <script> tags
- **Path Traversal:** Detects ../ navigation attempts

Key Feature: Real-time Performance Tracking

The dashboard displays exact execution time in milliseconds, proving KMP's efficiency advantage over naive algorithms.